Lecture 3 · Part I · Lecture notes

Classification and its metrics

Dataset: MNIST, detecting one digit

How to use these notes

The primary text for this lecture is Géron, Chapter 3. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The task, and a first detector

Two lectures on a regression problem ended with a number and an interval around it. Today the problem is classification, the metric is not RMSE, and the baseline matters more than it did in either of the previous two lectures.

1.1 The brief

A national postal operator sorts handwritten postcodes automatically: a camera photographs each digit, a classifier reads it, the envelope goes down a chute. Their audit team has found that one glyph is misread far more often than the others. They want every scanned digit that is a **5** pulled off the line and sent to a human verification desk.

Note what that is not. It is not "read the digit". It is *detect one digit* — yes or no, one class against the other nine.

Question	Answer
Supervision	supervised — every image carries a label
Task	binary classification: 5 or not-5
Learning	batch; the corpus is fixed and fits in memory
Assumption	next year's handwriting looks like this corpus

The fourth is the one that will age. Handwriting drifts, cameras are replaced, and a model trained on one decade's post is not automatically valid on the next.

1.2 What makes this a rare-event problem

The ten digits appear in roughly equal numbers. The *task* does not:

$$\text{one class against nine} \implies \text{about one positive in ten.}$$

This is not a fact about MNIST

Every binary detector carved out of a multiclass problem inherits this. Detect one product fault out of forty; one disease out of a hundred; one fraudulent transaction in ten thousand. The arithmetic is identical and only the ratio changes.
Nothing about MNIST is imbalanced. Our *question* is.

1.3 The data, and the first default nobody asked for

MNIST: 70,000 images of handwritten digits, each 28×28 pixels flattened to 784 features, each feature one pixel's intensity from 0 (white) to 255 (black). The split is standard — the first 60,000 rows train, the last 10,000 test — and using it is what makes any published MNIST number comparable to ours.

The labels arrive as one-character *strings*: `'5'`, not `5`.

Failure condition — what forgetting the cast costs

`y == 5` is then `False` for every image, with no error and no warning, because a string is never equal to an integer. You would build a detector for a class with no members and score 100% accuracy on it.
The repair is one line and one assertion: `y = y.astype(np.uint8)`, then `assert y_train_5.sum() == 5421`. If the cast was skipped, the assertion fails at 0 = 5421 instead of silently training on nothing.

1.4 Metric before model

Choose what you are optimising before you have anything to optimise — otherwise the metric gets chosen after the result, which is how you end up reporting whichever number happens to look best. And name who chose it: if the metric arrived by default, from a library or a habit or an assistant, then nobody chose it.

The obvious metric is **accuracy**, the fraction of instances the classifier gets right. It is between 0 and 1, bigger is better, it needs no threshold and no scaling, and every stakeholder understands it without being taught.

An `SGDClassifier` behind a `StandardScaler`, three-fold cross-validated, scores **96.90%** — folds of 96.83%, 96.84% and 97.04%, agreeing to about two tenths of a point.

1.5 Before you commit: what does nothing score?

The cheapest possible detector looks at nothing and answers "not a 5" every time. It is right 54,579 times out of 60,000: **90.96%** accuracy, with no model, no fit and no features.

Detector	Cross-validated accuracy
Never fires — the anchor	90.96%
SGD on raw pixels	95.03%
SGD in a pipeline with scaling	96.90%
Perfect	100%

Our detector is 5.94 percentage points above doing nothing and 3.10 points below perfect; of the mistakes the anchor makes we have removed 65.7%. Those are the same measurement stated two ways, and you should say which one you mean.

Read the anchor in two steps. First *what it is*: with no positive prediction, accuracy is exactly the negative class's base rate, an identity about the data with no model in it. Only then *what it means*: that accuracy is available for free, so 90.96% is zero in the units that matter. The anchor differs for every task — 88.76% for digit 1, because the corpus holds more 1s than any other digit.

2 Derivation: imbalance, and the non-monotonicity of precision

One question, for the whole lecture: doing nothing scores 90.96% and we score 96.90%. *What, exactly, did those 5.94 points buy?* The number is correctly cross-validated, there is no leakage in it, and the three folds agree to two tenths of a point. Everything about how it was measured is right, which is what makes the question worth twenty minutes.

2.1 Notation

m instances, each with a true label $y^{(i)} \in \{0, 1\}$; P positives, N negatives, $P + N = m$; $p = P/m$ the **base rate**. A scoring function s and a threshold t, predicting positive when $s(x) \geq t$, with counts $\mathrm{TP}(t)$, $\mathrm{FP}(t)$, $\mathrm{FN}(t)$, $\mathrm{TN}(t)$.

Everything below is arithmetic on those four counts. There is no probability theory in this derivation and none is needed.

2.2 Claim 1 — accuracy is a weighted average of two rates

$$\mathrm{accuracy}(t) \;=\; \frac{\mathrm{TP}(t) + \mathrm{TN}(t)}{m}.$$

Two of the four counts in the numerator, all four in the denominator — and note that the two mistakes enter with *equal weight*. A missed 5 and a wrongly flagged 3 cost the same, as far as this formula is concerned. Nobody in the postal operator believes that.

The decomposition

Split the four counts by true class, writing recall = TP/P and specificity = TN/N:

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{m} = \frac{P}{m} \cdot \frac{\text{TP}}{P} + \frac{N}{m} \cdot \frac{\text{TN}}{N}$$

$$= p\,\text{recall} + (1 - p)\,\text{specificity}.$$

The classifier that never fires is the special case recall = 0, specificity = 1, giving

$$\text{accuracy}(h_0) \;=\; \frac{0 + N}{m} \;=\; 1 - p.$$

An identity, not an estimate: no model, no data, no variance. On our task $p = 0.09035$, so accuracy$(h_0) = 0.90965$. That is the 90.96% we measured, and it could have been written down without running anything.

Failure condition — accuracy is unbounded above by uselessness

$$\lim_{p \to 0^+} \text{accuracy}(h_0) \;=\; 1.$$

For any target accuracy $a < 1$ you care to name, there is a detection problem rare enough that the empty model beats it.
So "99.2% accurate" is not a statement about a classifier. It is a statement about a classifier *and* a base rate, and quoting it without the second is quoting half a measurement.

The identity, on our own detector

Term	Value	Contribution
$(1 - p) \cdot$ specificity	0.90965×0.98860	0.89928
$p \cdot$ recall	0.09035×0.77181	0.06973
accuracy	—	0.96902

92.8% of our headline number is the first row — being right about the digits that are *not* 5s. Everything the detector does about the class we were hired to find is the 0.0697 on the second row.

Stated as a derivative:

$$\frac{\partial\,\text{accuracy}}{\partial\,\text{recall}} = p, \qquad \frac{\partial\,\text{accuracy}}{\partial\,\text{specificity}} = 1 - p.$$

Accuracy responds to the class you care about in proportion to *how little of it there is*. You can lose a third of the 5s and pay under three points of accuracy for it — and if accuracy is what you optimise, that trade looks attractive.

2.3 Claim 2 — recall is monotone in the threshold and precision is not

Every scoring classifier is really a family of classifiers, one per threshold: $\hat{y}_t(x) = \mathbf{1}[s(x) \geq t]$. Raising t makes it more reluctant to fire. Nothing about the model changes — only where we cut.

The flagged set $\{x : s(x) \geq t\}$ shrinks as t rises, and it shrinks by losing instances, never by gaining any. So for $t' \geq t$, both $\mathrm{TP}(t') \leq \mathrm{TP}(t)$ and $\mathrm{FP}(t') \leq \mathrm{FP}(t)$.

$$\mathrm{recall}(t) = \frac{\mathrm{TP}(t)}{\mathrm{TP}(t) + \mathrm{FN}(t)} = \frac{\mathrm{TP}(t)}{P}.$$

The denominator does not depend on t at all: TP + FN counts every positive in the data, whatever we predict about it. So recall is a non-increasing quantity divided by a constant, and recall is non-increasing. There are no exceptions and no conditions.

Precision has no such guarantee:

$$\mathrm{precision}(t) = \frac{\mathrm{TP}(t)}{\mathrm{TP}(t) + \mathrm{FP}(t)},$$

where numerator and denominator both move, in the same direction. A ratio of two decreasing quantities can do anything at all. The denominator is the size of the set *we chose to flag*: not a property of the data, a property of our decision.

The one-step lemma

Raise t just enough to drop the lowest-scoring flagged instance. Write $T = \mathrm{TP}$, $F = \mathrm{FP}$, $n = T + F$ before the step, and let $y \in \{0, 1\}$ be the true label of the instance dropped.

$$\text{precision before} = \frac{T}{n}, \qquad \text{precision after} = \frac{T - y}{n - 1}.$$

Cross-multiplying, the step increases precision exactly when $n(T - y) > T(n - 1)$, that is when

$$y < \frac{T}{n} = \text{precision before}.$$

Since y is 0 or 1, the lemma resolves into two cases:

Instance dropped	y	**Effect on precision**
a false positive	0	rises — always, since precision > 0
a true positive	1	falls — unless precision was already 1

The whole result in one sentence

Raising the threshold past a positive lowers precision; raising it past a negative raises precision. The non-monotonicity is not an anomaly — it is the generic case. Recall meanwhile falls by $1/P$ in the first case and by nothing in the second, so both metrics are decided by the same single fact: what the dropped instance was.

Counted on sixty thousand instances

Sort every training instance by its out-of-fold score and walk the threshold down the list one instance at a time — 59,999 steps.

As the threshold rises, one step at a time	Steps
the instance dropped is not a 5 — precision rises	54,579
the instance dropped is a 5 — precision falls	5,417
. . . or stays at 1, because it was already 1	3

54,579 is the number of non-5s in the training set, and 5,417 + 3 = 5,420 is the number of 5s less the one at the very top of the ranking, which has no step above it. Every step is accounted for by the lemma. This is not a tendency; it is an exact classification of all 59,999 steps.

What is guaranteed, then: recall is non-increasing, always; the flagged count is non-increasing, always; and precision has no monotone envelope you can rely on, only a local rule. That is worth remembering because *average precision* is defined using the maximum precision at or above each recall level, and that maximum exists for exactly one reason — to replace a non-monotone quantity by a monotone one.

3 What 96.90% was hiding

Detector	Accuracy	Recall
Never fires	90.96%	0%
Ours	96.90%	77.18%
Difference	5.94 points	77.18 points

The identity says where the 5.94 points came from. Recall earns $p \times 77.18\% = 6.97$ points; specificity fell from a perfect 100% to 98.86%, which costs $(1 - p) \times 1.14\% = 1.04$ points. The three numbers on the page are each rounded to two decimals, which is why they do not quite close: exactly, $6.9733 - 1.0367 = 5.9366$ against a measured difference of 5.9367.

3.1 The confusion matrix

	Predicted not-5	Predicted 5
Actually not-5	53,957	622
Actually 5	1,237	4,184

Rows are the actual class, columns the predicted class; the diagonal is what went right. Both mistakes are visible *separately*, and they are not the same size or the same kind. Accuracy added them together and divided by 60,000.

Metric	Value	Reads as
Precision	87.06%	of the digits we flag, 87% really are 5s
Recall	77.18%	of the 5s that exist, we find 77%
Accuracy	96.90%	—

1,237 fives went past unflagged: 22.8% of every 5 in the training set. Both of those numbers were computable an hour ago. Nothing new was fitted; we asked the same predictions a different question.

Failure condition — two sentences, both true

"The detector is 96.9% accurate." True. Signed off. Deployed.
"The detector misses 22.8% of the 5s — on the test shift, about 204 envelopes." Also true.
Same model, same data, same predictions.
Only one of them is a report the audit team can act on, and it is not the one the default metric produces.

Accuracy was not *wrong*. It correctly answered a question — what fraction of all decisions were right? — that nobody in this brief had asked. It weights the class we care about by $p = 0.09$, adds two errors the client prices completely differently, and has a floor of 90.96% that no model can go below by being cautious. And we chose it deliberately, before we had anything to optimise. Choosing early is necessary and not sufficient: only the class counts would have said it was the wrong metric.

4 Precision, recall, and the curves

4.1 Ask for both numbers, and know what F_1 costs

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Recall is also called **sensitivity** or the **true positive rate**; three names, one quantity, and the third returns on the ROC curve.

When one number is unavoidable, F_1 is the harmonic mean:

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Ours is 81.82%, below both numbers it combines. That is not an accident:

Precision	Recall	Arithmetic mean	F_1
0.90	0.90	0.900	0.900
0.99	0.81	0.900	0.891
1.00	0.02	0.510	0.039

The harmonic mean is dominated by the smaller of the two. A classifier that flags three digits a year and is right every time has an arithmetic mean above one half and an F_1 near zero.

But that is a choice, not a virtue

F_1 rewards classifiers with *similar* precision and recall. Our brief does not ask for similar precision and recall. It asks for the 5s to be found, within a fixed desk capacity.
A filter for children's videos wants high precision and low recall: rejecting a good video is annoying, letting a bad one through is not. A shoplifter detector wants high recall and accepts false alarms: a guard checking one costs a minute, a missed thief costs the stock.
Ours is the second shape, with a cap.

4.2 Where the dial is

Every scoring classifier computes a number and compares it to a threshold. `predict()` hides the number and hard-codes the threshold at zero. There is no `set_threshold()` method, and there should not be: you compute the scores with `decision_function` and compare them yourself.

Choosing 90% precision on cross-validated training scores gives threshold 20.09, at precision 0.9001 and recall 0.7333 — ninety per cent precision costs 3.86 points of recall, from 77.18% down to 73.33%.

Failure condition — why the crossing needs a support test

We have just spent twenty minutes proving the precision curve is not monotone. So "the first index reaching 90%" could be a single lucky step, held up by a handful of flagged digits, that the very next threshold falls back below.

Requiring the crossing to rest on at least 500 flagged digits turns it into a claim you can defend — and here it passes unchanged, at threshold 20.09 on 4,416 digits across 4,411 thresholds.

Aiming higher is nearly free-looking and very expensive. At 99% precision our recall falls to 2.12%: a detector that finds one 5 in fifty. And it fails the support test — 99% precision is reached on only 116 flagged digits, across a plateau 21 thresholds wide. Which is why 2.12% is quoted as an illustration of what high precision costs and never proposed as an operating point. The right response to a target you cannot meet with support is to *say so*, not to lower the support test until it passes.

A threshold is a hyperparameter, so it is chosen on cross-validated training scores, never on the test set — the same error as choosing α that way, with the same optimistic bias.

4.3 ROC, and when not to use it

Same sweep, different axes:

$$\text{TPR} = \frac{\text{TP}}{P} = \text{recall}, \qquad \text{FPR} = \frac{\text{FP}}{N} = 1 - \text{specificity}.$$

Ours scores ROC AUC 0.9724, against 0.5 for a coin and 1.0 for a perfect classifier.

Note that *both* denominators are fixed by the data. That is precisely why the ROC curve behaves so much better than the PR curve — and why that is a warning rather than a recommendation.

ROC AUC is the probability that a randomly chosen positive is scored above a randomly chosen negative. It is a statement about the ranking and nothing else: it does not depend on the threshold, because there is no threshold in it, and it does not depend on the base rate, because both axes are normalised within a class. So it cannot tell you whether the classifier is *usable*. Only how well it ranks.

Measure that, rather than taking it on trust

Take our scores unchanged and thin the positive class. The model, the ranking and the scores are identical; only the class balance moves.

Base rate	Positives	ROC AUC	Average precision
9.04% — as measured	5,421	0.9724	0.8784
2.00%	1,114	0.9699	0.7203
1.00%	551	0.9709	0.5800

ROC AUC moves by 0.0015. Average precision falls by 0.298. Three indistinguishable ROC curves and three obviously different PR curves, from one set of scores: *the ROC curve cannot see the problem you have*.

Hence the rule: prefer the precision/recall curve whenever the positive class is rare, or whenever you care more about false positives than false negatives. The reason is arithmetic — FPR divides false positives by N, which is huge when positives are rare, so a large number of false alarms produces a small, comfortable-looking FPR, while precision divides the same false positives by the flagged count, which is small.

4.4 And now a better classifier

A forest has no `decision_function`. It has `predict_proba`, whose positive-class column plays exactly the role of the score, and every curve above works unchanged.

Metric	SGD	Random forest
Accuracy	96.90%	98.77%
Precision	87.06%	98.93%
Recall	77.18%	87.31%
F_1	81.82%	92.76%
ROC AUC	0.9724	0.9984
Average precision	0.8784	0.9883
Recall available at 90% precision	73.33%	97.51%

Which metric noticed?

Accuracy improves by 1.87 points, which reads as polish. Recall improves by 10.13 points — the thing we care about. Recall at 90% precision improves by 24.18 points, and those are two completely different systems.

Three descriptions of one comparison. Which one you report decides whether anyone approves the change. Report the one that matches the brief, and say why you chose it. *That sentence is the deliverable.*

5 Choosing an operating point

What the client asked for, in our vocabulary: catch at least 90% of the 5s on a shift, and flag no more than 1,000 items out of 10,000 scanned. Two constraints, one dial — and the honest answer may be that no threshold satisfies both.

The forest at threshold 0.44 was chosen on cross-validated training scores, where it gives 90.39% recall at 98.33% precision. Measured once on the test shift:

Random forest at threshold 0.44	Recall	Precision	Flagged
Cross-validated, on the training set	90.39%	98.33%	—
Test shift, measured once	89.91%	98.89%	811

Half a point below target. Is that a failure, a bad fold, or noise? It is noise: the target was hit on a different sample, and a threshold chosen at exactly 90% will land either side of it about half the time.

Failure condition — tuning until the test number looks right

Re-tuning to make the test number reach 90% is fitting the test set — the temptation Lecture 2 told you to refuse, arriving exactly where it was predicted to.

Whichever threshold you propose, you must be able to answer four questions: which constraint did you satisfy and which did you sacrifice; what does one extra point of recall cost in items sent to the desk; on what data was the threshold chosen, and had you seen the test shift when you chose it; and what happens if the base rate changes next quarter.

The general rule

Never optimise one metric of a pair — precision and recall, bias and variance, latency and accuracy, memory and throughput. Ask for one and you will get it, at the price of the other, and the price will not be in the reply. The counter-measure is not cleverness; it is stating the second quantity as a constraint in the same sentence as the first.

6 Beyond binary

Everything above was one class against the rest. Three more shapes reuse the same confusion matrix.

Shape	Output per instance	Example
Binary	one label from two	is it a 5?
Multiclass	one label from many	which digit is it?
Multilabel	several labels at once	is it large? is it odd?
Multioutput	several labels, each multiclass	the denoised image, pixel by pixel

Multiclass is built from binary parts. **One-versus-rest** trains ten detectors like ours and takes the highest score. **One-versus-one** trains one detector per pair — $10 \times 9/2 = 45$ fits, each on a much smaller training set — and takes the most-won duel. Scikit-learn picks one for you automatically and does not mention which: another default nobody asked for, and it matters, because for algorithms that scale badly with training-set size forty-five small fits beat ten large ones.

For multilabel targets, metrics are computed per label and averaged, and the averaging argument is a third default that changes the number without changing the model: `average="macro"` weights every label equally, `average="weighted"` weights by support, and on imbalanced labels the two can disagree substantially. Name the metric, and name the average.

7 The notebook

What to change

Run it once, then re-run the base-rate experiment with the positive class thinned to 0.5% instead of 1%. ROC AUC will barely move again; watch what average precision does. Two lines, and it is the whole argument for preferring the PR curve on rare events — measured by you rather than asserted here.

8 Exercises

1. A colleague reports 99.2% accuracy on a rare defect, correctly cross-validated. State one further quantity you would need, and why. [4]

2. A ranking classifier is evaluated on 10 instances, of which 4 are positive. Sorted by score, highest first, the true labels are 1, 1, 0, 1, 0, 0, 1, 0, 0, 0. Compute precision and recall when the top 4 are flagged and when the top 5 are flagged; then state, using the one-step lemma, whether precision rises or falls when the threshold is raised from "top 5" to "top 4", and why. [3]

3. Explain why F_1 would be the wrong summary for the postal operator's brief, and name the pair of numbers you would report instead. [4]

4. A model's ROC AUC is unchanged when the positive class is thinned from 9% to 1%, while its average precision falls by 0.30. Explain both observations in terms of the denominators of the two curves' axes. [5]

5. You are asked for a threshold achieving 99% precision. The first crossing rests on 116 flagged instances across a plateau 21 thresholds wide. State what you would report, and why lowering the support test until the target passes is not an option. [4]

Summary

Accuracy decomposes as $p\,\text{recall} + (1-p)\,\text{specificity}$, so a classifier that never fires scores exactly $1-p$ and $\partial\,\text{accuracy}/\partial\,\text{recall} = p$. On this task that anchor is 90.96%, our detector scored 96.90%, and 92.8% of the headline was being right about digits that are not 5s.

Recall is non-increasing in the threshold because its denominator is P, which the threshold cannot touch. Precision's denominator is the flagged count, which the threshold chooses, so one step raises precision exactly when the dropped instance is a negative — counted on our own ranking as 54,579 rises, 5,417 falls and 3 flat, every one of 59,999 steps accounted for.

Asking the same predictions a different question gave precision 87.06% and recall 77.18%: 1,237 fives past the desk, 22.8% of them. ROC AUC moved by 0.0015 as the base rate fell from 9% to 1% while average precision fell by 0.298, which is why the PR curve is the one to read on a rare event. And the forest improved accuracy by 1.87 points, recall by 10.13, and recall at 90% precision by 24.18 — three descriptions of one comparison, of which only the last matches the

brief.